

CENG467 Midterm

Tan Aksu

1 Question 1: Representation Learning in Text Classification

1.1 Objective

The objective of this experiment is to analyze how different representation learning strategies (sparse, dense, and contextual) affect text classification performance and generalization.

1.2 Dataset

The IMDb (Internet Movie Database) dataset is a widely used benchmark for binary sentiment classification. It consists of movie reviews labeled as either positive or negative, making it suitable for evaluating text classification models.

The dataset contains a total of 50,000 labeled reviews, split evenly into 25,000 training samples and 25,000 test samples. The class distribution is balanced, with an equal number of positive and negative examples, which ensures that accuracy is a reliable evaluation metric.

Each review is relatively long compared to other sentiment datasets, often containing multiple sentences and complex expressions. This makes the dataset particularly useful for evaluating models that need to capture long-range dependencies and contextual information.

In addition, the dataset includes a large vocabulary with diverse linguistic patterns, such as negation (e.g., “not good”), sarcasm, and mixed sentiment expressions. These characteristics make IMDb a challenging benchmark and a good testbed for comparing different representation learning approaches, including sparse, dense, and contextual models. The IMDb dataset was used. It contains 50,000 labeled movie reviews (positive/negative). We used an 80/20 split for training and validation, and the official test set for final evaluation.

1.3 Preprocessing and Tokenization

Text normalization was applied using lowercase transformation.

TF-IDF: Two configurations were tested:

- Stopwords removed
- Stopwords kept

BiLSTM: Word-level tokenization + padding and truncation.

BERT: WordPiece subword tokenization with max length 256.

1.4 Models

TF-IDF + Logistic Regression: Sparse representation baseline.

BiLSTM: Sequence-based neural model.

DistilBERT: Pretrained transformer-based contextual model.

Model	Val Acc	Val F1	Test Acc	Test F1
TF-IDF (stopwords removed)	0.8930	0.8930	0.8794	0.8794
TF-IDF (stopwords kept)	0.8972	0.8972	0.8911	0.8911
TF-IDF (no lowercase)	0.8914	0.8914	0.8882	0.8882
BiLSTM (256)	0.8504	0.8503	0.8225	0.8225
BiLSTM (128)	0.8374	0.8374	0.8079	0.8079
DistilBERT	0.9116	0.9116	0.9115	0.9115

Table 1: Model performance comparison

1.5 Results

1.6 Error Analysis

To better understand model behavior, misclassified validation examples were analyzed for all three main representation types: TF-IDF, BiLSTM, and DistilBERT. The analysis shows that most errors are not random; they are commonly caused by mixed sentiment, implicit sentiment, misleading sentiment keywords, long reviews, and domain-specific expressions.

Review Snippet	True	Pred.	Model	Error Pattern
“This is a must see for independent movie fans...”	1	0	TF-IDF	Positive sentiment is expressed through recommendation-style language, but the model fails to detect it.
“Yowsa! If you REALLY want some ACTION...”	1	0	TF-IDF / BiLSTM / DistilBERT	Exaggerated promotional language is misinterpreted as negative or low-quality writing.
“Despite its budget limitations, this is a great film...”	1	0	TF-IDF	Mixed wording: negative phrase “budget limitations” appears together with positive sentiment.
“I rated this a ten just because I find it so impressive...”	1	0	TF-IDF	Positive sentiment is implicit and depends on understanding the full sentence.
“This movie tackles child abduction...”	0	1	TF-IDF / BiLSTM / DistilBERT	Serious topic and neutral description are confused with positive sentiment.

Table 2: Representative misclassified validation examples. Label 0 indicates negative sentiment and label 1 indicates positive sentiment.

Several common error patterns were observed. First, all models struggled with exaggerated or promotional language. For example, the review beginning with “Yowsa! If you REALLY want some ACTION...” was incorrectly classified as negative by multiple models, although the true label was positive. This suggests that unusual writing style can mislead both classical and neural models.

Second, mixed sentiment was a frequent source of error. Reviews containing both negative and positive phrases, such as “Despite its budget limitations, this is a great film”, were difficult to classify correctly. TF-IDF is especially vulnerable to this because it relies on word and n-gram statistics rather than full contextual meaning.

Third, some errors were caused by implicit sentiment. In examples such as “I rated this a ten...”, the positive meaning is clear to a human reader but may not be captured by the model if sentiment is not expressed through simple polarity words. This also explains why narrative-style reviews, such as the Star Trek-related example misclassified by DistilBERT, remain difficult.

Fourth, domain-specific or strongly polarized words created confusion. For instance, the BiLSTM and DistilBERT misclassified the review containing words such as “PIGs” and “power-mad” as negative, even though the reviewer was positively evaluating the movie. This shows that even contextual models can be

sensitive to strongly negative lexical cues.

Finally, long reviews were challenging because sentiment information may appear in different parts of the text. This is particularly important for BiLSTM and DistilBERT, where truncation limits the amount of input text processed by the model. Overall, the error analysis shows that although DistilBERT achieves the best quantitative performance, all models still struggle with nuanced, implicit, and mixed sentiment expressions.

1.7 Discussion

The results clearly demonstrate the impact of representation learning strategies on both performance and interpretability. Among all models, the transformer-based DistilBERT achieved the highest performance, highlighting the advantage of contextual representations. Its ability to model word meaning dynamically based on context allows it to better handle complex linguistic patterns such as negation, long-range dependencies, and mixed sentiment expressions.

The TF-IDF model, representing sparse representations, provided a strong and competitive baseline. Despite its simplicity, it achieved relatively high performance and proved to be highly interpretable. Feature importance can be directly analyzed through word and n-gram weights, making it easier to understand model decisions. However, TF-IDF lacks the ability to capture word order and contextual meaning, which limits its performance compared to transformer-based models.

The BiLSTM model, representing dense sequence-based representations, demonstrated moderate performance. While it is capable of capturing sequential dependencies, it suffers from limited ability to model long-range context and is more sensitive to truncation. Additionally, since it was trained from scratch without pretrained embeddings, its generalization ability was weaker compared to BERT.

In terms of interpretability, TF-IDF is the most transparent model, followed by BiLSTM, which offers limited interpretability due to dense hidden representations. DistilBERT, although the most powerful in terms of performance, is the least interpretable due to its complex architecture and attention mechanisms.

Overall, the results highlight a trade-off between performance and interpretability. Sparse models are simple and interpretable but limited in expressiveness, dense models provide richer representations but require more data and tuning, while contextual models achieve the best performance at the cost of higher complexity and lower interpretability.

1.8 Conclusion

This study demonstrates that representation choice plays a crucial role in text classification performance. While classical methods such as TF-IDF remain strong and efficient baselines, they lack the ability to capture contextual meaning. Neural sequence models such as BiLSTM introduce sequential modeling but still struggle with long-range dependencies and generalization.

Transformer-based models such as DistilBERT achieve the best overall performance due to their ability to leverage pretrained knowledge and capture contextual relationships. However, this comes at the cost of higher computational complexity and lower interpretability.

Additionally, the experiments show that preprocessing decisions such as stopword removal, lowercasing, and truncation can significantly influence performance. Therefore, both model selection and preprocessing design should be carefully considered in real-world NLP applications.

2 Question 2: Named Entity Recognition

2.1 Objective

The objective of this task is to examine sequence labeling and contextual modeling in named entity recognition (NER). Specifically, classical feature-based models and transformer-based models are compared in terms of performance and error behavior.

2.2 Dataset and Annotation Structure

The CoNLL-2003 dataset was used as the benchmark dataset for named entity recognition. It consists of sentences annotated with named entity labels at the token level. Each token is associated with a tag following the BIO (Beginning, Inside, Outside) scheme.

The dataset includes four main entity types:

- PER (Person)
- ORG (Organization)
- LOC (Location)
- MISC (Miscellaneous)

The BIO tagging scheme is defined as follows:

- B-XXX: Beginning of an entity
- I-XXX: Inside an entity
- O: Outside any entity

For example:

```
EU rejects German call
B-ORG O B-MISC O
```

Proper alignment between tokens and labels was ensured. For the transformer-based model, token-label alignment was handled carefully by assigning labels only to the first subword token and ignoring the rest using a masking value.

2.3 Models

Two different modeling approaches were implemented and compared:

2.3.1 Conditional Random Field (CRF)

A classical sequence labeling model based on Conditional Random Fields was implemented. The model uses handcrafted features such as word identity, suffixes, capitalization, and surrounding context (previous and next tokens). CRF models are effective in modeling label dependencies but rely heavily on feature engineering.

2.3.2 DistilBERT for Token Classification

A transformer-based model was implemented using DistilBERT. The model was fine-tuned for token classification using contextual embeddings. Unlike CRF, DistilBERT does not rely on manual feature design and instead learns representations directly from data using self-attention mechanisms.

2.4 Experimental Setup

All models were trained and evaluated on the same dataset splits. Precision, Recall, and F1-score were used as evaluation metrics. A fixed random seed was used for reproducibility.

For the CRF model, handcrafted features were extracted for each token. For the DistilBERT model, tokenization was performed using WordPiece tokenization, and sequences were truncated to a maximum length of 128 tokens.

Model	Precision	Recall	F1-score
CRF (Validation)	0.8931	0.8578	0.8751
CRF (Test)	0.8140	0.7716	0.7922
DistilBERT (Validation)	0.9222	0.9355	0.9288
DistilBERT (Test)	0.8795	0.8967	0.8880

Table 3: NER performance comparison

2.5 Results

The results show that DistilBERT significantly outperforms the CRF model across all evaluation metrics. The improvement is particularly noticeable in recall, indicating that the transformer-based model is better at detecting named entities.

2.6 Error Analysis

To better understand model behavior, misclassified examples from both models were analyzed. Several common error patterns were identified.

Entity Confusion. Both models sometimes confuse entity types. For example, the token “CHINA” was incorrectly classified as a location instead of a person in a specific context, showing reliance on common lexical usage.

Boundary Errors. The CRF model frequently struggles with multi-token entities. For instance, in the phrase “World Cup”, the model incorrectly assigns multiple beginning tags instead of maintaining a consistent entity span.

Incorrect Entity Type Assignment. In some cases, entities are assigned the wrong category. For example, nationality-related terms such as “Uzbek” may be misclassified as persons instead of miscellaneous entities.

Contextual Errors. DistilBERT generally reduces errors compared to CRF, but it still struggles in semantically ambiguous cases where entity meaning depends heavily on context.

2.7 Discussion

The comparison highlights the importance of contextual modeling in NER. The CRF model relies on hand-crafted features and local context, which limits its ability to capture complex relationships between words. As a result, it struggles with boundary detection and entity type classification.

In contrast, DistilBERT leverages contextual embeddings, allowing it to model long-range dependencies and capture semantic meaning more effectively. This leads to significantly better performance, especially in recall.

However, despite its strong performance, DistilBERT is more computationally expensive and less interpretable than CRF. This demonstrates a trade-off between performance and model simplicity.

2.8 Conclusion

This study demonstrates that transformer-based models significantly improve named entity recognition performance compared to classical approaches. While CRF models provide a simple and interpretable baseline, they are limited by feature engineering and lack of contextual understanding.

DistilBERT achieves superior performance by leveraging pretrained contextual representations. However, both models still exhibit errors in boundary detection and entity classification, indicating that NER remains a challenging task, especially in complex linguistic contexts.

3 Question 3: Text Summarization

3.1 Objective

The objective of this task is to compare extractive and abstractive summarization methods. In this experiment, TextRank was used as the extractive summarization method, while T5 was used as the abstractive summarization model.

3.2 Dataset

The CNN/DailyMail dataset was used for this task. It consists of news articles paired with human-written summaries called highlights. Each sample contains a long news article as the input text and a reference summary as the target output.

Due to computational constraints, a subset of 50 test samples was used for evaluation. This subset was used consistently for both TextRank and T5 summaries.

Property	Description
Dataset	CNN/DailyMail
Input	News article
Reference output	Human-written highlights
Task	Text summarization
Subset size	50 test samples

Table 4: Key properties of the CNN/DailyMail dataset.

3.3 Methods

3.3.1 Extractive Summarization with TextRank

TextRank was implemented as the extractive summarization method. The article was first split into sentences. Sentence representations were created using TF-IDF, and cosine similarity was used to build a similarity graph between sentences. PageRank was then applied to rank the sentences, and the top-ranked sentences were selected as the final summary.

Since TextRank selects sentences directly from the original article, it is generally faithful to the source text. However, it may produce summaries that are less fluent or contain unnecessary details.

3.3.2 Abstractive Summarization with T5

T5-small was used as the abstractive summarization model. Unlike TextRank, T5 generates new summary sentences instead of selecting existing sentences from the source article. The input text was prefixed with the task instruction summarize and summaries were generated using beam search.

T5 can produce more fluent and natural summaries, but it may omit important information or generate content that is not fully supported by the source article.

3.4 Evaluation Metrics

The generated summaries were evaluated using a comprehensive set of metrics.

- **ROUGE-1**: Measures unigram overlap between generated and reference summaries.
- **ROUGE-2**: Measures bigram overlap.
- **ROUGE-L**: Measures longest common subsequence overlap.
- **BLEU**: Measures n-gram precision, commonly used in machine translation.
- **METEOR**: Uses alignment, stemming, and synonym matching.

- **BERTScore**: Measures contextual semantic similarity using pretrained language models.

3.5 Quantitative Results

Model	ROUGE-1	ROUGE-2	ROUGE-L	BLEU	METEOR	BERTScore F1
TextRank	0.2667	0.0862	0.1829	0.0559	0.2966	0.8597
T5	0.3152	0.1296	0.2386	0.0952	0.3064	0.8673

Table 5: Quantitative comparison of TextRank and T5 summarization methods.

The results show that T5 outperforms TextRank across all automatic evaluation metrics. The improvement in ROUGE and BLEU indicates better lexical overlap with the reference summaries, while the higher BERTScore suggests stronger semantic similarity.

3.6 Qualitative Analysis

Three qualitative examples were analyzed to compare fluency, factual consistency, and information coverage.

3.6.1 Example 1

In the first example, the article discusses the Palestinian Authority becoming a member of the International Criminal Court. TextRank selected factual sentences directly from the article, including official statements and quotations. This made the summary factually consistent, but somewhat long and less focused.

T5 generated a shorter and more fluent summary. However, it omitted an important detail from the reference summary: the opposition from Israel and the United States. This shows that although T5 improves readability, it may lose some information coverage.

3.6.2 Example 2

In the second example, the article describes the story of Theia, a dog that survived after being hit by a car, struck with a hammer, and buried. TextRank focused on veterinary treatment and fundraising details rather than the most important dramatic events. As a result, its summary was factual but did not fully capture the main point of the article.

T5 produced a more readable and coherent summary, but it also missed some important details such as the hammer incident and burial. This suggests that abstractive models can improve fluency but may still fail to cover all essential facts.

3.6.3 Example 3

In the third example, the article discusses Mohammad Javad Zarif and details about his political background. TextRank selected some relevant factual sentences, but the summary was not very coherent and focused on secondary details.

T5 produced a more fluent output, but it included less relevant information and showed signs of factual inconsistency. This demonstrates a potential weakness of abstractive summarization: the generated text may sound natural even when it does not fully reflect the source article.

3.7 Discussion

The results demonstrate clear trade-offs between extractive and abstractive summarization. TextRank is computationally efficient and factually reliable because it directly extracts sentences from the source article. However, its summaries may be less fluent, less concise, and sometimes include irrelevant details.

T5 achieved better automatic metric scores and produced more fluent summaries. This shows the advantage of abstractive summarization in generating human-like summaries. However, qualitative examples showed that T5 may omit important information or produce summaries with factual inconsistencies.

In terms of computational cost, TextRank is much cheaper because it does not require a large pretrained neural model. T5 is more computationally expensive due to transformer-based generation, but it provides better readability and stronger semantic similarity to reference summaries.

3.8 Conclusion

This experiment compared extractive and abstractive summarization using TextRank and T5 on a subset of the CNN/DailyMail dataset. T5 achieved better performance across ROUGE, BLEU, METEOR, and BERTScore, indicating stronger similarity to human-written summaries.

However, the qualitative analysis shows that better automatic scores do not always guarantee perfect factual consistency or information coverage. TextRank remains useful when faithfulness to the source is important, while T5 is more suitable when fluency and readability are prioritized. Overall, the choice between extractive and abstractive summarization depends on the desired trade-off between computational cost, readability, and factual faithfulness.

4 Question 4: Machine Translation

4.1 Objective

The objective of this task is to analyze sequence-to-sequence modeling and attention mechanisms in machine translation. A Seq2Seq model with attention was implemented and compared with a pretrained Transformer-based translation model.

4.2 Dataset

The OPUS Books English-German dataset was used for this task. Although Multi30k is a common benchmark for machine translation, OPUS Books was selected due to its stable availability and aligned English-German sentence pairs.

Each sample consists of a source sentence in English and a reference translation in German. A subset of the dataset was used to make the experiments computationally feasible.

Property	Description
Dataset	OPUS Books
Task	English-to-German machine translation
Source language	English
Target language	German
Training samples	2400
Validation samples	300
Test samples	300
Evaluation subset	100 test samples

Table 6: Dataset properties for the machine translation task.

4.3 Preprocessing

For the Seq2Seq model, preprocessing included lowercasing, punctuation separation, whitespace normalization, tokenization, vocabulary construction, truncation, and padding. Special tokens were added to represent padding, unknown words, and sentence boundaries: `<pad>`, `<unk>`, `<sos>`, and `<eos>`.

The maximum sequence length was set to 30 tokens. The source vocabulary contained 3212 tokens, while the target vocabulary contained 3617 tokens. For the pretrained Transformer model, preprocessing was handled by the Marian tokenizer, which applies subword tokenization and truncation automatically.

4.4 Models

4.4.1 Seq2Seq with Attention

A Seq2Seq model with attention was implemented using an encoder-decoder architecture. The encoder reads the English source sentence and produces hidden representations. The attention mechanism allows the decoder to focus on different source tokens while generating each German target token.

The model used embedding layers, GRU-based encoder and decoder modules, and an additive attention mechanism. It was trained from scratch for 5 epochs using teacher forcing.

4.4.2 Pretrained Transformer Model

A pretrained MarianMT Transformer model, `Helsinki-NLP/opus-mt-en-de`, was used as the Transformer-based alternative. Unlike the Seq2Seq model, this model was already pretrained on large-scale parallel corpora and directly used for translation inference.

4.5 Experimental Setup

The Seq2Seq model was trained on 2400 sentence pairs and validated on 300 sentence pairs. The model was trained for 5 epochs using the Adam optimizer. Translation quality was evaluated on 100 test examples.

Hyperparameter	Value
Random seed	42
Source vocabulary size	3212
Target vocabulary size	3617
Maximum sequence length	30
Embedding dimension	128
Hidden dimension	256
Batch size	32
Epochs	5
Optimizer	Adam

Table 7: Experimental setup for the Seq2Seq model.

4.6 Training Behavior

The Seq2Seq model showed decreasing training and validation loss during training. This indicates that the model learned some translation patterns, although the overall translation quality remained limited.

Epoch	Train Loss	Validation Loss
1	5.9043	5.1682
2	5.2936	5.0651
3	5.0281	5.0021
4	4.7972	4.9816
5	4.5379	4.9763

Table 8: Seq2Seq training and validation loss.

4.7 Evaluation Metrics

The translation outputs were evaluated using four metrics:

- **BLEU**: Measures n-gram precision with brevity penalty.
- **METEOR**: Measures alignment quality and accounts for stemming and synonym matching.

- **ChrF**: Character n-gram F-score, useful for morphologically rich languages such as German.
- **BERTScore**: Measures contextual semantic similarity using pretrained language models.

4.8 Quantitative Results

Model	BLEU	METEOR	ChrF	BERTScore F1
Seq2Seq + Attention	0.0000	0.0663	4.3236	0.5260
Pretrained Transformer	0.1343	0.3962	41.6727	0.8078

Table 9: Machine translation performance comparison.

The pretrained Transformer model outperformed the Seq2Seq model across all metrics. The Seq2Seq model received a BLEU score of 0.0 because it failed to produce sufficient higher-order n-gram overlap with the reference translations.

4.9 Qualitative Analysis

Source	Reference	Seq2Seq Output	Transformer Output
From then on she never failed to open the door slightly every evening and morning and look briefly in on him.	Seitdem versäumte sie nicht, stets flüchtig morgens und abends die Tür ein wenig zu öffnen und zu Gregor hineinzuschauen.	sie sich in ihrem manne, und <unk>, und <unk>, und <unk>	Von da an hat sie es nie versäumt, jeden Abend und jeden Morgen die Tür leicht zu öffnen und ihn kurz anzuschauen.
Let's!	Komm doch!	wie! rief <unk> ...	Auf geht's!
And I've got some things to tell you that I've learned about.	Auch ich habe dir Verschiedenes mitzuteilen, was ich erfahren habe.	und du sie, daß sie, daß sie ...	Und ich muss dir einiges sagen, worüber ich gelernt habe.

Table 10: Qualitative examples of machine translation outputs.

The qualitative examples show that the Seq2Seq model often produces repetitive phrases and unknown tokens. This indicates limited vocabulary coverage and weak generalization. In contrast, the pretrained Transformer model generates fluent and semantically meaningful German translations.

The Transformer model handles word order, rare words, and longer dependencies more effectively. The Seq2Seq model struggles with these cases because it was trained from scratch on a small subset and has no pretrained linguistic knowledge.

4.10 Discussion

The results demonstrate a clear performance gap between the Seq2Seq model with attention and the pretrained Transformer model. Although the attention mechanism helps the Seq2Seq decoder focus on relevant source tokens, the model still struggles due to limited training data, small vocabulary size, and lack of pretraining.

BLEU reflects the low n-gram overlap of the Seq2Seq outputs, which explains the score of 0.0. METEOR also shows that Seq2Seq translations have poor lexical and alignment quality. ChrF strongly favors the Transformer model, indicating better character-level similarity, which is especially important for German morphology. BERTScore further confirms that the Transformer model preserves semantic meaning much better than the Seq2Seq model.

The pretrained Transformer model benefits from large-scale pretraining and subword tokenization. This allows it to handle rare words and produce fluent translations. However, it is more computationally expensive and less interpretable than the smaller Seq2Seq model.

4.11 Conclusion

This experiment shows that pretrained Transformer-based translation models significantly outperform Seq2Seq models trained from scratch. The Seq2Seq model with attention demonstrates the basic mechanism of encoder-decoder translation, but its performance is limited by data size, vocabulary coverage, and training time.

The Transformer model provides better fluency, semantic preservation, and handling of rare words or long-range dependencies. However, this improvement comes at the cost of higher computational complexity. Overall, the comparison highlights the practical advantage of pretrained Transformer models for machine translation tasks.

5 Question 5: Language Modeling

5.1 Objective

The objective of this task is to investigate probabilistic sequence modeling and text generation. A trigram language model was implemented and evaluated using perplexity. In addition, short text samples were generated to analyze fluency, coherence, and model behavior.

5.2 Dataset

The WikiText-2 dataset was used for this task. WikiText-2 consists of Wikipedia-style articles and is commonly used for language modeling experiments. The dataset provides separate training, validation, and test splits.

Split	Number of Sentences Used
Train	19,430
Validation	2,075
Test	2,351

Table 11: Dataset statistics after preprocessing.

5.3 Preprocessing

The text was lowercased and tokenized using a simple rule-based tokenizer. Punctuation symbols were separated from words, and empty lines or very short sequences were removed. Sentence boundary tokens were added using two start tokens `<s>` and one end token `</s>`.

Rare words were handled using an unknown token `<unk>`. Words appearing fewer than two times in the training data were replaced with `<unk>`. After preprocessing, the vocabulary size was 38,896.

5.4 Model

A trigram language model was implemented. The model estimates the probability of each word based on the previous two tokens:

$$P(w_i \mid w_{i-2}, w_{i-1})$$

Laplace smoothing was used to avoid zero probabilities for unseen trigrams:

$$P(w_i \mid w_{i-2}, w_{i-1}) = \frac{C(w_{i-2}, w_{i-1}, w_i) + 1}{C(w_{i-2}, w_{i-1}) + |V|}$$

where C denotes count values and $|V|$ is the vocabulary size.

The model produced 617,053 trigram contexts from the training data.

5.5 Evaluation

The model was evaluated using perplexity. Perplexity measures how well a language model predicts a sequence. Lower perplexity indicates better predictive performance.

$$\text{Perplexity} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i) \right)$$

Split	Perplexity
Validation	16663.71
Test	16413.69

Table 12: Perplexity results of the trigram language model.

The perplexity values are high, indicating that the trigram model has limited predictive ability. This is expected because the model only conditions on the previous two tokens and cannot capture long-range semantic dependencies.

5.6 Generated Text Samples

Generated Samples
modesty, of the 21st digital content grand prix. the battle of orton a itself, which concluded on 27 march 1871 in offenburg charged <unk> with the rest of
because of inherent <unk> of nme called “heroes”, which features a line, to a gilded ship whose occupants are unaware of her translation of english football
thoth, as in most inflammatory responses, transcription, post’s stuever has many unusual features unusual headgear; a life-giving function, study abroad experiences
spanish hill reportedly refused to be displayed at the nippon budokan in 2014; it was received that rabaul, ambon and timor. the grown ups plan to reverse
ransome argues that some ceratopsians were the most dramatic and recalled many important thanouser pictures, stated the track list, and moved from june on, ch. 31

Table 13: Generated text samples from the trigram language model.

5.7 Analysis

The generated samples show that the trigram model can produce short local word patterns that sometimes resemble natural language. For example, phrases such as “the battle of” or “which features” are grammatically plausible. This shows that the model learns local co-occurrence patterns from the training corpus.

However, the generated text lacks global coherence. Topics shift abruptly, and the model often combines unrelated concepts in the same sentence. This happens because the trigram model only uses the previous two words as context and does not maintain a longer semantic representation of the text.

The model also produces <unk> tokens, showing limitations in vocabulary coverage. Rare or unseen words cannot be generated properly and are replaced with the unknown token. This reduces both fluency and readability.

Overall, the trigram model demonstrates the basic idea of probabilistic language modeling, but its ability to generate fluent and coherent text is limited compared to neural or transformer-based language models.

5.8 Conclusion

This experiment shows that n-gram language models can capture local word dependencies but struggle with long-range coherence and semantic consistency. The high perplexity values indicate that the trigram model has difficulty predicting diverse WikiText-2 sequences.

Although the generated samples contain some grammatically plausible fragments, they are often incoherent and topic-inconsistent. More advanced models such as LSTM or transformer-based language models would likely achieve lower perplexity and generate more fluent and coherent text by modeling longer contexts.

Source Code

The implementation and all experiment files are available on GitHub:

[GitHub Repository Link](#)